ЗАДАНИЕ 1

```
from itertools import *
c = '345 367 1267 156 147 234 235'.split()
b = 'ЕЖ ЕВ ВА АБ БЖ ЖД ДБ ДГ ДЕ ГВ ГА'.split()
print('1 2 3 4 5 6 7')
for p in permutations('АБВГДЕЖ'):
    if all(str(p.index(x)+1) in c[p.index(y)] for x,y in b):
        print(*p)
```

ЗАДАНИЕ 2

¬A	not()	не A (отрицание, инверсия)
A ∧ B	and	A и B (умножение, конъюнкция)
A ∨ B	or	A или B (сложение, дизъюнкция)
A → B	<=	импликация (следование)
A ≡ B	==	эквивалентность (равносильность)
Закон импликации:	A → B = ¬ A ∨ B	
Формулы Де Моргана:	¬(A∧B) = ¬A∨¬B или ¬(A∨B) = ¬A∧¬B	

```
print('x y z w')
for x in 0,1:
   for y in 0,1:
        for z in 0,1:
            for w in 0,1:
                if not(  (w and y) or ((x<=w)==(y<=z))  ):
                    print(x,y,z,w)
```

```
from itertools import *
def f(x,y,z,w):
    return ((z == x) <= w) and (w <= (y and x))
for x1,x2,x3 in product([0,1],repeat=3):
    t = ((1, 1, x1, 0, 1),
         (1, x2, x3, 0, 1),
         (1, 0, 1, 1, 1))
    if len(t) == len(set(t)):
        for p in permutations('xyzw', 4):
            if all(f(**dict(zip(p,l))) == l[-1] for l in t):
                print(*p)
```

ЗАДАНИЕ 3

=ВПР(искомое значение(ключ); таблица поиска; номер столбца в диапазоне с возвращаемым значением; возврат приблизительного или точного совпадения - указывается как 1 или 0).
=ВПР(C2; Альбомы!$A$2:$C$348; 3; 0)
НЕ ЗАБУДЬ ЗАФИКСИРОВАТЬ ТАБЛИЦУ ПОИСКА!

ЗАДАНИЕ 4

Прямое условие Фано: Никакое кодовое слово не может быть началом другого кодового слова.
Обратное условие Фано: никакой код не был окончанием другого (более длинного) кода.
Внимательно читай, нужно ли использовать весь алфавит! Если нужно, то оставь одно место!

ЗАДАНИЕ 5

f{x:b}" представление числа x в двоичной системе
oct(x)[2:] представление числа x в восьмеричной системе
hex(x)[2:] представление числа x в шестнадцатеричной системе
int(x, n) перевод числа x (в формате строки) в 10 из n системы счисления

ШАБЛОН ЗАДАЧИ

```
for n in range(1, 100):
    n2 = bin(n)[2:]
    if n % 2 == 0:
        n2 += n2[-2:]
    else:
        n2 = '1' + n2 + '0'
    r = int(n2, 2)
    if r < 100:
        print(n)
```

САМОПИСНАЯ ФУНКЦИЯ ПЕРЕВОДА

```
def troi(n):
    s = ''
    while n > 0:
        s = str(n%3) + s
        n //= 3
    return s
```

ЗАДАНИЕ 6

Повтори 7 [Налево 60 Вперёд 5 Налево 120 Вперёд 5]

КУМИР

```
использовать Черепаха
алг
нач
опустить хвост
нц 7 раз
влево(60)
вперед(5)
влево(120)
вперед(5)
кц
кон
```

X ∩ Y или пересечение X + Y

ПИТОН

```
from turtle import *
const = 30
tracer(0)
lt(90)
screensize(2000, 2000)
rt(45)
for i in range(7):
    fd(5*const)
    rt(45)
    fd(10*const)
    rt(135)

up()
for x in range(-20, 20):
    for y in range(-20, 30):
        goto(x*const, y*const)
        dot(3, "blue")
update()
exitonclick()
```

X ∪ Y или объединение X + Y

ЗАДАНИЕ 7

1 байт = 8 бит = 2^3 бит
1 Кбайт = 2^{10} байт= $2^{10} \cdot 2^3$ бит = 2^{13} бит
1 Мбайт = 2^{10} Кбайт = $2^{10} \cdot 2^{10}$ байт = 2^{20} байт = $2^{20} \cdot 2^3$ бит = 2^{23} бит.

Для хранения растрового изображения нужно выделить в памяти

V = M · N · i

битов, где M ширина, N высота и i – глубина цвета.

N = 2^I – количество цветов.

Для хранения информации о звуке длительностью t секунд, закодированном с частотой дискретизации f Гц и глубиной кодирования I бит и количестве каналов k требуется

V = f · t · i · k бит.

Например, при стерео записи (k = 2), f = 32кГц, глубине кодирования 16 бит и длительности звука 2 минуты требуется = (2·32000·16·(2·60)) / (2**23) ≈ 14,6 Мбайт.

При сжатии за 100% берем то, с чем сравниваем!

ЗАДАНИЕ 8

Перестановки	$P_n = n!$
Перестановки с повтор.	$\overline{P}_{k(n1,n2...nk)} = \frac{n!}{n1! * n2! * ... * nk!}$
Размещения	$A_n^k = \frac{n!}{(n-k)!}$
Размещения с повтор.	$A_n^k = n^k$
Сочетания	$C_n^k = \frac{n!}{(n-k)! * k!}$
Сочетания с повтор.	$\overline{C}_n^k = \frac{(k+n-1)!}{(n-1)! * k!}$

Петя составляет шестибуквенные слова перестановкой букв слова КАБАЛА. При этом он избегает слов с двумя подряд одинаковыми буквами.

```
from itertools import permutations
s = set()
for i in permutations('КАБАЛА', 6):
    a = ''.join(i)
    if 'АА' not in a:
        s.add(a)
print(len(s))
```

Все пятибуквенные слова, в составе которых могут быть только русские буквы Л, А, Й, М, записаны в алфавитном порядке и пронумерованы начиная с 1. Под каким номером в списке идёт последнее слово, которое содержит не более одной буквы М и не содержит букв Л, стоящих рядом?

```
from itertools import *
alf = sorted('ЛАЙМ')
cnt = 0
for i in product(alf, repeat=5):
    s = ''.join(i)
    cnt += 1
    if s.count('M') <= 1 and 'ЛЛ' not in s:
        print(cnt, s)
```

Примечание: Число не может начинаться с 0! 0 чётное число.

ЗАДАНИЕ 9

Используй функции НАИБОЛЬШИЙ(), НАИМЕНЬШИЙ() для сортировки чисел в строке.

В файле в каждой строке записаны семь натуральных чисел. Определите количество строк таблицы, для чисел которых выполнены оба условия:
– в строке есть одно число, которое повторяется трижды, остальные четыре числа различны;
– среднее арифметическое неповторяющихся чисел строки не больше повторяющегося числа.

```
f = open('9.txt')
cnt = 0
for s in f:
    a = [int(x) for x in s.split()]
    p = [x for x in a if a.count(x) == 3]
    np = [x for x in a if a.count(x) == 1]
    if len(p) == 3 and len(np) == 4:
        srnp = sum(np) / len(np)
        if srnp <= p[0]:
            cnt += 1
print(cnt)
```

ЗАДАНИЕ 11

На предприятии каждой изготовленной детали присваивается серийный номер, содержащий десятичные цифры, 26 латинских букв (без учёта регистра) и символы из 450-символьного специального алфавита. В базе данных для хранения каждого серийного номера отведено одинаковое и минимально возможное число байт. При этом используется посимвольное кодирование серийных номеров, все символы кодируются одинаковым и минимально возможным числом бит. Известно, что для хранения 708 серийных номеров отведено более 213 Кбайт памяти. Определите минимально возможную длину серийного номера. В ответе запишите только целое число.

```
from math import *
a = 10 + 26 + 450
b = ceil(log2(a))
for l in range(1, 1000):
    v = ceil((l * b)/8)
    if 708 * v > 213 * 1024:
        print(l)
        break
```

На предприятии каждой изготовленной детали присваивают серийный номер, состоящий из 261 символов. Для его хранения отведено одинаковое и минимально возможное число байт. При этом используется посимвольное кодирование серийных номеров, все символы кодируются одинаковым и минимально возможным числом бит. Известно, что для хранения 252 500 серийных номеров отведено более 31 Мбайт памяти. Определите минимально возможную мощность алфавита, из которого составляются серийные номера. В ответе запишите только число.

```
from math import *
for a in range(1, 10000):
    b = ceil(log2(a))
    v = ceil((261 * b)/8)
    if 252500 * v > 31 * 1024 * 1024:
        print(a)
        break
```

ЗАДАНИЕ 13

Сеть задана IP-адресом 10.48.96.0 и маской сети 255.255.240.0. Сколько в этой сети IP-адресов, для которых количество единиц в двоичной записи IP-адреса больше, чем количество нулей? В ответе укажите только число.

```
from ipaddress import *
net = ip_network('10.48.96.0/255.255.240.0', 0)
cnt = 0
for ad in net:
    ad2 = f'{ip:b}'    # ad2 = bin(int(ip))[2:].zfill(32)
    if ad2.count('1') > ad2.count('0'):
        cnt += 1
print(cnt)
```

Сеть задана IP-адресом одного из входящих в неё узлов 98.81.154.195 и сетевой маской 255.252.0.0.
Найдите наибольший в данной сети IP-адрес, который может быть назначен компьютеру. В ответе укажите найденный IP-адрес без разделителей.
Например, если бы найденный адрес был равен 111.22.3.44, то в ответе следовало бы записать 11122344

```
from ipaddress import *
net = ip_network(f'98.81.154.195/255.252.0.0', 0)
print(net[-2])
net[-1] – Широковещательный адрес
net[0] – Адрес сети
```

ЗАДАНИЕ 14

Операнды арифметического выражения записаны в системе счисления с основанием 15:

$$123x5_{15} + 1x233_{15}$$

В записи чисел переменной x обозначена неизвестная цифра из алфавита 15-ричной системы счисления. Определите наименьшее значение x, при котором значение данного арифметического выражения кратно 14. Для найденного значения x вычислите частное от деления значения арифметического выражения на 14 и укажите его в ответе в десятичной системе счисления.

```
for x in '0123456789abcde':
    c = int('123'+x+'5', 15) + int('1'+x+'233', 15)
    if c%14==0:
        print(c//14)
        break
```

Значение выражения $(729^{41} - 81^{16}) \cdot (729^{15} + 9^5)$ записали в системе счисления с основанием 9. Сколько цифр 0 содержится в этой записи?

```
x = (729**41 - 81**16)*(729**15 + 9**5)
cnt0 = 0
while x > 0:
    if x%9 == 0:
        cnt0 += 1
    x //= 9
print(cnt0)
```

Значение арифметического выражения

$$4^{210} + 4^{110} - x,$$

где x – целое положительное число, не превышающее 3000, записали в четверичной системе счисления. Определите наименьшее значение x, при котором в четверичной записи числа, являющегося значением данного арифметического выражения, содержится наибольшее количество нулей.

```
mx = 0
for x in range(1,3001):
    cnt = 0
    c = 4**210 + 4**110 - x
    while c > 0:
        if c%4 == 0:
            cnt += 1
        c//= 4
    if cnt > mx:
        mx = cnt
        print(x)
```

ЗАДАНИЕ 15

На числовой прямой даны два отрезка: B = [14; 20] и C = [15; 27]. Укажите наименьшую возможную длину такого отрезка A, для которого логическое выражение ¬(x ∈ A) → ((x ∈ B) ≡ (x ∈ C)) истинно (т. е. принимает значение 1) при любом значении переменной x.

```
def f(x):
    b = 14<=x<=20
    c = 15<=x<=27
    a = a1<=x<=a2
    return (not a) <= (b == c)
d = [p for x in [14,15,20,27] for p in [x-0.01, x, x+0.01]]
res = []
for a1 in d:
    for a2 in d:
        if a2>a1 and all(f(x)==1 for x in d):
            res.append(a2-a1)
print(round(min(res)))
```

Обозначим через ДЕЛ(n, m) утверждение «натуральное число n делится без остатка на натуральное число m». Для какого наибольшего натурального числа A формула ДЕЛ(x, 32) → (¬ДЕЛ(x, A) → ¬ДЕЛ(x, 12))
тождественно истинна (то есть принимает значение 1 при любом натуральном значении переменной x)?

```
def f(x,a):
    return (x%32==0) <= ((x%a!=0) <= (x%12!=0))
for a in range(1,1000):
    if all(f(x,a) for x in range(1,1000)):
        print(a)
```

Укажите наименьшее целое значение A, при котором выражение (5x + 3y ≠ 60) ∨ ((A > x) ∧ (A > y))
истинно для любых целых неотрицательных значений x и y.

```
def f(x,y,a):
    return (5*x + 3*y != 60) or ((a > x) and (a > y))
for a in range(1000):
    if all(f(x,y,a) for x in range(1000) for y in range(1000)):
        print(a)
        break
```

Для какого наименьшего неотрицательного целого числа A формула
(x & 29 ≠ 0) → ((x & 17 = 0) → (x & A ≠ 0)) тождественно истинна (т.е. принимает значение 1 при любом неотрицательном целом значении переменной x)?

```
def f(x,a):
    return (x&29!=0) <= ((x&17==0) <= (x&a!=0))
for a in range(1000):
    if all(f(x,a) for x in range(1000)):
        print(a)
        break
```

ЗАДАНИЕ 16

Sys.setrecursionlimit(5000) – увеличение глубины рекурсии

Алгоритм вычисления функции F(n), где n – натуральное число, задан следующими соотношениями:

F(n) = n, если n < 11,
F(n) = n + F(n-1), если n ≥ 11.

Чему равно значение выражения F(2024) – F(2021)?

```
import sys
sys.setrecursionlimit(5000)
def f(n):
    if n < 11: return n
    if n >= 11: return n + f(n-1)
print(f(2024)-f(2021))
#или
from functools import lru_cache
@lru_cache(50)
def f(n):
    if n < 11: return n
    if n >= 11: return n + f(n-1)
for i in range(2024): f(i)
print(f(2024)-f(2021))
#или
a = [0]*2025
for n in range(0,2025):
    if n < 11: a[n] == n
    if n >= 11: a[n] = n + a[n-1]
print(a[2024]-a[2021])
```

ЗАДАНИЕ 17

Элементы последовательности могут принимать целые значения от –10 000 до 10 000 включительно. Определите количество пар, в которых хотя бы один из двух элементов заканчивается на 17, а их сумма делится на 2.

```
f = open('17.txt')
a = [int(s) for s in f]
res = []
for i in range(len(a) - 1):
    x = a[i]
    y = a[i + 1]
    if (abs(x) % 100 == 17) + (abs(y) % 100 == 17) >= 1:
        if (x + y) % 2 == 0:
            res.append(x+y)
print(len(res), min(res))
```

В файле содержится последовательность целых чисел, не превышающих по модулю 100 000. Определите количество троек элементов последовательности, в которых не более двух из трёх элементов являются четырёхзначными числами, а сумма элементов тройки не больше максимального элемента последовательности, оканчивающегося на 25. В ответе запишите количество найденных троек чисел, затем максимальную из сумм элементов таких троек.

```
f = open('17.txt')
a = [int(s) for s in f]
res = []
m2 = max(x for x in a if abs(x)%100==25)
for i in range(len(a) - 2):
    x = a[i]
    y = a[i + 1]
    z = a[i + 2]
    if (1000<=abs(x)<=9999) + \
            (1000<=abs(y)<=9999) +\
            (1000<=abs(z)<=9999) <= 2:
        if x + y + z <= m25:
            res.append(x+y+z)
print(len(res), max(res))
```

ЗАДАНИЕ 18

$A1 – Зафиксированный столбец, A$1 – Зафиксированная строка, $A$1 – все зафиксированно*

(Стандартные ходы) Определите максимальную и минимальную денежную сумму, которую может собрать Робот, пройдя из **левой верхней** клетки в **правую нижнюю**

=МИН(J22;K21) + J10

672	670	619
637	703	
682	723	=МИН(J22;K21) + J10

Формула для ладьи
(ходит на любое количество клеток по вертикали и горизонтали)

=МАКС($Q2:Q2; R$1:R1)+C2

8	16	-1	3	32
12	-14	-4	20	37
11	-12	-6	-2	35
39	23	30	46	47

ЗАДАНИЯ 19-21

1 куча камней, ходы +3 *2

```
from functools import *
@lru_cache(None)
def g(h):
    m = [h + 3, h * 2]
    if h >= 69:
        return 'w'
    if any(g(x) == 'w' for x in m):
        return 'p1'
    if all(g(x) == 'p1' for x in m):
        return 'v1'
    if any(g(x) == 'v1' for x in m):
        return 'p2'
    if all(g(x) == 'p1' or g(x) == "p2"
for x in m):
        return 'v1orv2'

for s in range(1, 69):
    if g(s) == 'v1':
        print(s)
19задание в кавычке пишем  v1
20заданиb в кавычке пишем  p2
19задание в кавычке пишем  v1orv2
```

2 кучи камней, ходы +1 *2

```
from functools import *
@lru_cache(None)
def g(h):
    a, b = h  # Создаем два числа, (две
кучи)
    m = [(a + 1, b), (a, b + 1), (a * 2,
b), (a, b * 2)]  # Всевозможные ходы
    if a + b >= 259:
        return "w" # Сумма при которой
засчитывается побера
    if any(g(x) == 'w' for x in m):
        return 'p1'  # Петя поб 1 ч
    if all(g(x) == 'p1' for x in m):
        return "v1"  # Ваня поб после
хода пети
    if any(g(x) == 'v1' for x in m):
        return 'p2'  # Петя поб 2 х
    if all(g(x) == "p1" or g(x) == "p2"
for x in m):
        return "v1orv2"  # ваня поб или 1
или 2 ходом


for s in range(1, 241+1):
    h = 17, s  # наши камни из задания
    if g(h) == "v1orv2":  # меняем в
зависимости от задания
        print(s)

не забываем all менять на any когда
неудачный ход
```

ЗАДАНИЕ 22

Формула времени окончания процесса + сдвиг

E2 =МАКС(ВПР(C2;A:E;5;0);ВПР(D2;A:E;5;0))+B2+G2

	A	B	C	D	E	F	G
1	0				0		
2	101	14	0	0	14	1	

Формула времени старта

F2 =E2-B2+1

	A	B	C	D	E	F	G
1	0				0		
2	101	14	0	0	14	1	

Формула отрисовки диаграммы Ганта

J2 =ЕСЛИ(И(J$1>=$F2;J$1<=$E2);1;"")

	A	B	C	D	E	F	G	H	I	J
1	0				0					1
2	101	14	0	0	14	1			101	1

ЗАДАНИЕ 23

У исполнителя есть три команды, которым присвоены номера:
1. Прибавить 1 2. Прибавить 2 2. Умножить на 3
Сколько существует программ, для которых при исходном числе 2 результатом является число 12, и при этом траектория вычислений содержит числа 8 и 10?

```
def f(x, y):
    if x > y:
        return 0
    if x == y:
        return 1
    return f(x + 1, y) + f(x + 2, y) + f(x * 3, y)
print(f(2,8)*f(8,10)*f(10,12))
```

Знаки > и < меняются на противоположные при отрицательных ходах

У исполнителя есть две команды, которым присвоены номера:
1. Прибавь 1 2. Сделай нечетное
Сколько различных результатов можно получить из исходного числа 5 после выполнения программы, содержащей ровно 15 команд?

```
s = set()
def f(x, cnt):
    if cnt == 15:
        s.add(x)
        return
    f(x+1,cnt+1)
    f(x*2+1,cnt+1)
f(5,0)
print(len(s))
```

ЗАДАНИЕ 24

s.find(str)	Поиск подстроки str в строке s. Возвращает номер первого вхождения или -1
s.rfind(str)	Поиск подстроки str в строке s с конца. Возвращает номер последнего вхождения или -1
s.count('A')	Количество A в строке s
s.replace('old', 'new', cnt)	Замена всех подстрок «old» на «new» необходимое количество раз cnt
s.split('C')	Превращает строку в список строк, разделяя по символу, в кавычках
S[start:stop:step]	Извлечение среза [старт: стоп(невключит): шаг]

Текстовый файл состоит из символов, обозначающих буквы латинского алфавита A, B и C и цифры 8 и 9. Определите в прилагаемом файле максимальное количество идущих подряд символов, среди которых ни одна буква не стоит рядом с буквой, а цифра — с цифрой.

```
s = open('24.txt').readline().strip()
s = s.replace('B', 'A').replace('C', 'A')
s = s.replace('9', '8')
while 'AA' in s or '88' in s:
    s = s.replace('88', '8 8')
    s = s.replace('AA', 'A A')
s = s.split()
print(max(len(x) for x in s))
```

Текстовый файл состоит из десятичных цифр и заглавных букв латинского алфавита. Определите в этом файле последовательность идущих подряд символов, представляющих собой запись максимального нечётного 12-ричного числа. В ответе запишите индекс (номер) первого символа (первой значащей цифры), с которого начинается запись этого числа в прилагаемом файле. Нумерация символов в текстовом файле начинается с нуля.

```
s = open('24.txt').readline()
l = ''; mx = 0
for e in s:
    if len(l)==0 and e=='0': continue
    elif e in '0123456789AB':
        l += e
        if int(l,12)%2!=0 and int(l,12)>mx:
            mx = int(l,12); ans = l
    else: l = ''
print(s.find(ans))
```

Текстовый файл 6. txt состоит из заглавных букв латинского алфавита A, B, C, D, E и F.

Определите в прилагаемом файле максимальное количество идущих подряд символов, среди которых пара AB (в указанном порядке) встречается ровно 100 раз. Для выполнения этого задания следует написать программу.

```
s = open('6.txt').readline().strip()
s = s.replace('AB', 'A B')
s = s.split()
maxl = 0
for i in range(len(s)):
    maxl = max(sum(len(x) for x in s[i:i+101]), maxl)
print(maxl)
```

Текстовый файл состоит не более чем из 10^6 символов и содержит только десятичные цифры, а также знаки «+» и «*» (сложения и умножения). Определите максимальное количество символов в непрерывной последовательности, являющейся корректным арифметическим выражением с целыми неотрицательными числами (без знака), **значение которого равно нулю**. В этом выражении никакие два знака арифметических операций не стоят рядом, порядок действий определяется по правилам математики. В записи чисел отсутствуют незначащие (ведущие) нули. В ответе укажите количество символов в найденном выражении.

```
from re import *
s = open('7.txt').readline()
ch = r'([0]|[1-9][0-9]*)'
p = rf'({ch}[*])*0([*]{ch})*'
reg = rf'{p}([+]{p})+'
print(max(len(x.group()) for x in finditer(reg,s)))
print(max([x.group() for x in finditer(reg,s)], key=len))
```

Текстовый файл состоит из десятичных цифр и заглавных букв латинского алфавита. Определите в прилагаемом файле максимальное количество идущих подряд символов, среди которых подстрока 2025 встречается не менее 90 раз и при этом содержится ровно 80 букв Y. В ответе запишите число - количество символов в найденной последовательности.

```
s = open('7.txt').readline().split('Y')
m = 0
for i in range(len(s)):
    c = s[i:i+81]
    c = ''.join(c)
    if c.count('2025') >= 90:
        m = max(m, len(c)+80)
print(m)
#или
s = open('7.txt').readline()
m = 0
for l in range(len(s)):
    for r in range(l+m, len(s)):
        c = s[l:r+1]
        if c.count('Y')==80 and c.count('2025')>=90:
            m = max(m, len(c))
        if c.count('Y')>80: break
print(m)
```

Текстовый файл состоит не более чем из 10^6 символов и содержит только заглавные буквы латинского алфавита. Определите минимальную длину подстроки, в которой символ Z встречается не менее 120 раз.

```
a = open('3.txt').readline().split('Z')
minl = 10**10
for i in range(len(a)-119):
    minl = min(minl, sum(len(x) for x in a[i:i+119])+120)
print(minl)
```

ЗАДАНИЕ 25

Пусть F - разность максимального и минимального натуральных делителей, не считая 1 и самого числа. Если таких делителей у числа нет, то считаем значение F=0. Напишите программу, которая перебирает числа, большие 850 000, и ищет среди них те, для которых значение F не равно нулю и делится нацело на 13.

```
def F(n):
    d = set()
    for i in range(2, int(n**0.5) +1):
        if n % i == 0:
            d.add(i)
            d.add(n//i)
    if d:
        return max(d) - min(d)
    return 0

for n in range(850_001, 851_001):
    f = F(n)
    if f > 0 and f % 13 == 0:
        print(n, f)
```

Напишите программу, которая перебирает целые числа, большие 15 381 264, в порядке возрастания и ищет среди них числа, представленные в виде произведения ровно трёх простых множителей, не обязательно различных, каждый из которых содержит в своей записи ровно одну цифру 1.

```
def dell(n):
    m = []
    for d in range(2,round(n**0.5)+1):
        while n%d==0:
            m.append(d)
            n //= d
    if n != 1: m.append(n)
    return m
for n in range(15381265, 15383600):
    m = dell(n)
    if len(m)==3 and
all(str(x).count('1')==1 for x in m):
        print(n, max(m))
```

Среди натуральных чисел, не превышающих 1010, найдите все числа, соответствующие маске 89*6?7?9?, делящиеся на 9874 без остатка.

```
from fnmatch import fnmatch
for i in range(0, 10**10+1, 9874):
    if fnmatch(str(i), '89*6?7?9?'):
        print(i, i//9874)
```

Среди натуральных чисел, не превышающих 1013, найдите все числа, соответствующие маске 12?345?67089?, делящиеся на 206 без остатка.

```
from itertools import product
for x,y,z in product('0123456789',
repeat=3):
    c = int('12'+x+'345'+y+'67089'+z)
    if c%206==0:
        print(c, c//206)
```

символ «#» означает любую последовательность четных цифр произвольной длины; в том числе «#» может задавать и пустую последовательность.
— символ «?» означает одну любую цифру. Найдите все натуральные числа, которые не превышают 10^{11}, соответствуют маске 123#45?67 и делятся на 257

```
from itertools import product
for r in range(4):
    for i in product('02468',
repeat=r):
        for v in range(10):
            c =
int('123'+''.join(i)+'45'+str(v)+'67')
            if c%257 == 0:
                print(c, c//257)
```

ЗАДАНИЕ 26

Сортировка по возрастанию	a.sort()
Сортировка по убыванию	a.sort(reverse = True)
Сортировка по второму числу	a.sort(key=lambda i: i[1])

Примеры программ

Задача на коробки, которые складывают по принципу матрешки

```
f = open('26-3.txt')
n = int(f.readline())
a = []
for i in range(n):
    a.append(int(f.readline()))
a.sort(reverse = True)
res = 1
prev = a[0]
for i in range(1,n):
    if prev - a[i] >= 3:
        res += 1
        prev = a[i]
print(res, prev)
```

Задача про багажные ячейки на оптимизацию очереди

```
f = open('26-1.txt')
k = int(f.readline())
n = int(f.readline())
pas = []
for i in range(n):
    p, u = map(int,
f.readline().split())
    pas.append([p, u])
pas.sort()
cnt = maxt = maxn = 0
for number in range(1,k+1):
    busy = 0
    for x in pas:
        if x[0] - busy >= 1 and
x[0] != -1:
            cnt += 1
            busy = x[1]
            if x[0] > maxt:
                maxt = x[0]
            maxn = number
        x[0] = x[1] = -1
print(cnt, maxn)
```

Алгоритм сканлайн (уместить как можно больше мероприятий)

```
f = open('26-3.txt')
n = int(f.readline())
mer = []
for i in range(n):
    s, k = map(int,
f.readline().split())
    mer.append([s, k])
mer.sort(key = lambda x:x[1])
ans = [mer[0]]
for i in range(1,n):
    if mer[i][0] >= ans[-1][1]:
        ans.append(mer[i])
for i in range(1,n):
    if mer[i][0] >= ans[-2][1]:
        ans.pop(-1)
        ans.append(mer[i])
print(len(ans), ans[-1][1])
```

Шлифовка деталей (формальный алгоритм)

```
f = open('26-3.txt')
n = int(f.readline())
t = []
cnt = 0
for i in range(n):
    cnt += 1
    sh, ok = map(int,
f.readline().split())
    if sh < ok:
        t.append([sh, 'sh', cnt])
    else:
        t.append([ok, 'ok', cnt])
t.sort()
print(t[-1][2], len([x for x in
t[:-1] if x[1]=='sh']))
```

Магазин фиксирующий входы и выходы (самая затоптанная точка)

```
f = open('26_8.txt')
n = int(f.readline())
t = [0]*24*60
for i in range(n):
    s, e = map(int,
f.readline().split())
    for j in range(s,e+1):
        t[j] += 1
maxt = max(t)
t.append(0)
cnt = 0
for i in range(1,len(t)):
    if t[i] != maxt and t[i-1] ==
maxt:
        cnt += 1
print(cnt, maxt)
```

Матрица (лесополоса или места в кинотеатре)

```
f = open("26_2.txt")
n, k = map(int,
f.readline().split())
a = []
for i in range(n):
    x, y = map(int,
f.readline().split())
    a.append([x, y])
a.sort()
maxr = 0
for i in range(1, len(a)):
    if a[i][0] == a[i-1][0] and
a[i][1] - a[i-1][1] == 12:
        if i > maxr:
            maxr = a[i][0]
            minm = a[i-1][1] + 1
print(maxr, minm)
```

Загрузка диска (жадный алгоритм)

```
f = open("26.txt")
s, n = map(int,
f.readline().split())
a = []
for i in range(n):
    a.append(int(f.readline()))
a.sort()
b = []
for i in range(len(a)):
    if sum(b) + a[i] <= s:
        b.append(a[i])
    elif sum(b)-b[-1]+a[i]<=s:
        del b[-1]
        b.append(a[i])
    else:
        break
print(len(b), max(b))
```

Скидки и расположение товаров на ленте

```
f = open("26-1.txt")
n = int(f.readline())
a = []
for i in range(n):
    a.append(int(f.readline()))
a.sort()
pokup = magaz = 0
for i in range(n):
    if i <= 7499:
        pokup += a[i]
    else:
        pokup += a[i]/2
for i in range(n):
    if i <= 2499:
        magaz += a[i]/2
    else:
        magaz += a[i]
print(int(pokup), int(magaz))
```

ЗАДАНИЕ 27

Стандартная кластеризация (центроид или антицентр)
*антицентр – наоборот самая удаленная точка

```
from math import dist
f = open('../file27/27_2_Zvar3A.txt')
data = []
for s in f:
    s = s.replace(',', '.')
    p = [float(x) for x in s.split()]
    data.append(p)
print(len(data))
def get_cluster(p0):
    cluster = [p for p in data if dist(p, p0) <= 1]
    if cluster:
        for p in cluster:
            data.remove(p)
        new_clusters = [get_cluster(p) for p in cluster]
        cluster += sum(new_clusters, [])
    return cluster
clusters = []
while data:
    cluster = get_cluster(data[0])
    print(len(cluster))
    clusters.append(cluster)
def center(cluster):
    m = []
    for p0 in cluster:
        sm = sum(dist(p0, p) for p in cluster)
        m.append([sm, p0])
    return min(m)[1]
centers = [center(cluster) for cluster in clusters]
print(centers)
```

Задача с цветом светимостью и размером звезд (кластеризация с доп. вопросами)

A1 – минимальное расстояние от центра кластера до любого жёлтого гиганта в другом кластере, а A2 – максимальное расстояние от центра кластера до любого жёлтого гиганта в другом кластере.

Для файла Б определите координаты центра каждого кластера, затем найдите два числа:
B1 – минимальное расстояние между двумя различными жёлтыми сверхгигантами, расположенными в разных кластерах, и B2 – расстояние между центрами кластеров с минимальным и максимальным количеством жёлтых сверхгигантов.

```
from math import dist
from types import new_class

f = open('27_A_28766.txt')
data = []
for s in f:
    s = s.replace(',', '.')
    x, y, s = s.split()
    x = float(x)
    y = float(y)
    color = s[0]
    svet = int(s[1])
    razm = s[2:]
    p = [x, y, color, razm]
    data.append(p)

print(len(data), data[0])


def red_gigant(data):
    red = []
    for x, y, color, razm in data:
        if color == 'Y' and razm == 'III':
            red.append([x, y, color, razm])
    return red

red_gigants = red_gigant(data)


def get_cluster(p0):
    cluster = [p for p in data if dist(p0[:2], p[:2]) <= 1]
    if cluster:
        for p in cluster:
            data.remove(p)
        new_clusters = [get_cluster(p) for p in cluster]
        cluster += sum(new_clusters, [])
    return cluster


clusters = []
while data:
    cluster = get_cluster(data[0])
    print(len(cluster))
    clusters.append(cluster)


def center(cluster):
    m = []
    for p0 in cluster:
        sm = sum(dist(p[:2], p0[:2]) for p in cluster)
        m.append([sm, p0])
    return min(m)[1]


centers = [center(cluster) for cluster in clusters]
print(centers)
print(len(red_gigants))

dist_do_red_gig = [dist(centers[1][:2], p[:2]) for p in
red_gigants]
print(dist_do_red_gig)
A1 = min(dist_do_red_gig)
A2 = max(dist_do_red_gig)

print(int(A1*10_000), int(A2*10_000))

from turtle import *
from random import *
c = 30
tracer(0)
screensize(10_000, 10_000)
up()

for cluster in clusters:
    color = random(), random(), random()
    for x, y, cwet, razm in cluster:
        goto(x*c, y*c)
        dot(5, color)

for x, y, color, razm in centers:
    goto(x*c, y*c)
    dot(10, 'black')

for x, y, color, razm in red_gigants:
    goto(x * c, y * c)
    dot(10, 'red')

hideturtle()
done()
```

Задача с цветом светимостью и размером звезд (кластеризация с доп. вопросами)

A1 – минимальное расстояние от центра кластера до любого жёлтого гиганта в другом кластере, а A2 – максимальное расстояние от центра кластера до любого жёлтого гиганта в другом кластере.

Для файла Б определите координаты центра каждого кластера, затем найдите два числа:
B1 – минимальное расстояние между двумя различными жёлтыми сверхгигантами, расположенными в разных кластерах, и B2 – расстояние между центрами кластеров с минимальным и максимальным количеством жёлтых сверхгигантов.

```
from math import dist
from types import new_class

f = open('27_B_28766.txt')
data = []
for s in f:
    s = s.replace(',', '.')
    x, y, s = s.split()
    x = float(x)
    y = float(y)
    color = s[0]
    svet = int(s[1])
    razm = s[2:]
    p = [x, y, color, razm]
    data.append(p)

print(len(data), data[0])


def yellow_giant(data):
    yellow = []
    for x, y, color, razm in data:
        if color == 'Z' and razm == 'I':
            yellow.append([x, y, color, razm])
    return yellow

yellow_giants = yellow_giant(data)


def get_cluster(p0):
    cluster = [p for p in data if dist(p0[:2], p[:2]) <= 1]
    if cluster:
        for p in cluster:
            data.remove(p)
        new_clusters = [get_cluster(p) for p in cluster]
        cluster += sum(new_clusters, [])
    return cluster


clusters = []
while data:
    cluster = get_cluster(data[0])
    print(len(cluster))
    clusters.append(cluster)


def center(cluster):
    m = []
    for p0 in cluster:
        sm = sum(dist(p[:2], p0[:2]) for p in cluster)
        m.append([sm, p0])
    return min(m)[1]


centers = [center(cluster) for cluster in clusters]
print(centers)
print(len(yellow_giants))

def dist_for_yellow(yellow_giants):
    m = []
    for p0 in yellow_giants:
        d = [dist(p0[:2], p[:2]) for p in yellow_giants if p != p0]
        m.append(min(d))
    return min(m)

B1 = dist_for_yellow(yellow_giants)
print(B1)
B2 = dist(centers[0][:2], centers[1][:2])
print(B2)

print(int(B1*10_000), int(B2*10_000))

from turtle import *
from random import *
c = 30
tracer(0)
screensize(10_000, 10_000)
up()

for cluster in clusters:
    color = random(), random(), random()
    for x, y, cwet, razm in cluster:
        goto(x*c, y*c)
        dot(5, color)
    write(f'{len(cluster)}')

for x, y, color, razm in centers:
    goto(x*c, y*c)
    dot(10, 'black')

for x, y, color, razm in yellow_giants:
    goto(x * c, y * c)
    dot(10, 'yellow')

hideturtle()
done()
```